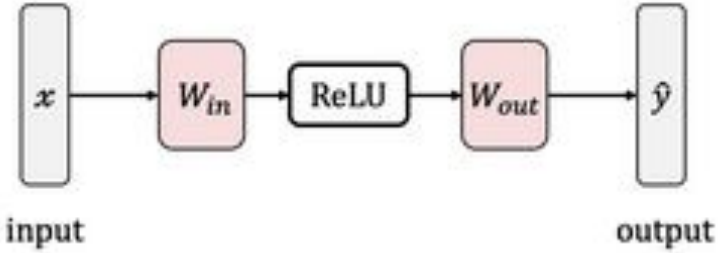
Activation Plateaus Meeting Slides

Francisco Ferreira da Silva and Stefan Heimersheim

Meeting 16/04/2026 - Contents

1. Computation in Superposition
2. (if there's time, otherwise Monday) writing plan

CiS - Model



With labels y = ReLU (x)

(so **no mixing matrix**)

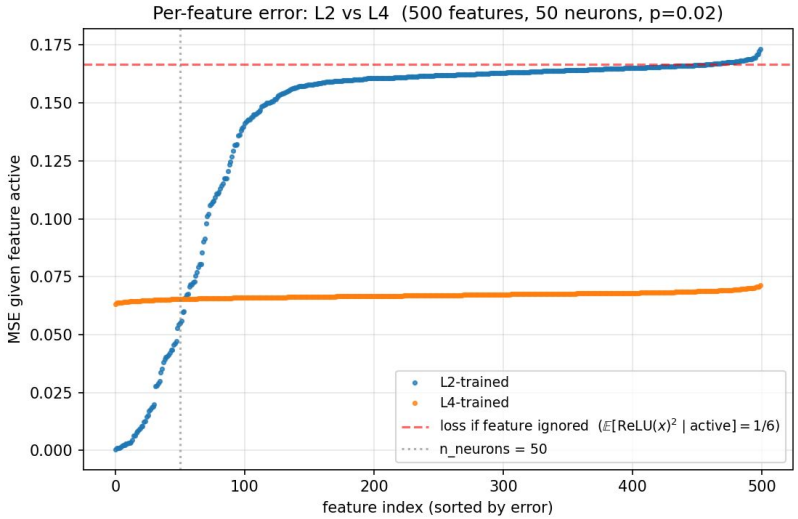
I think we need a task where the labels are somehow more discrete, or **the loss function punishes outlier errors** more, or the computation has multiple steps, where later steps in the computation depend on lots of intermediary results computed to low precision.

Lucius on LessWrong

Idea: L^4 loss rather than L^2

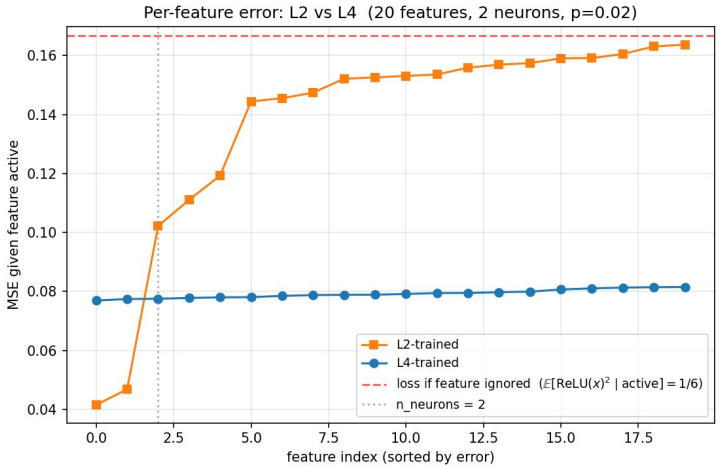
Model trained on L^2 and L^4

50 neurons, 500 features, p=0.02
Data uniform (-1,1)



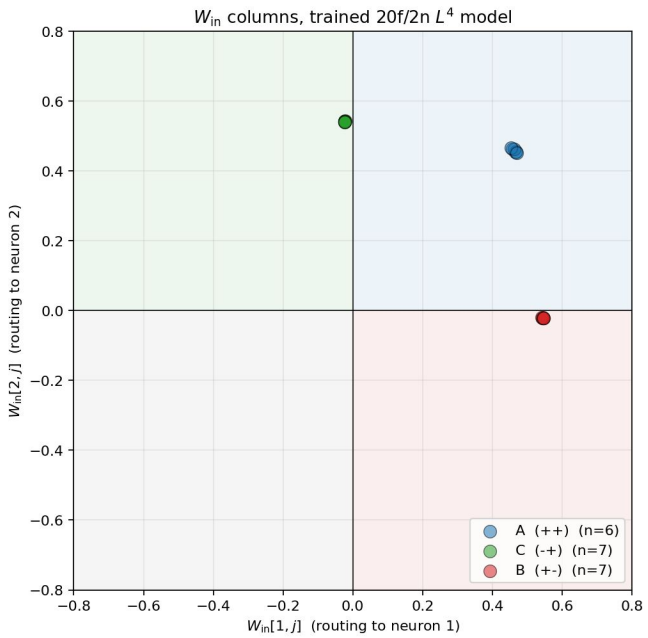
L^2 looks naive, L^4 treats features ~uniformly

Can we make our life easier?
What's the smallest model that apopears to do something smart?



Same plot for 2 neurons, 20 features
Can we reverse engineer this one?

Reverse-engineering small L^4-trained model

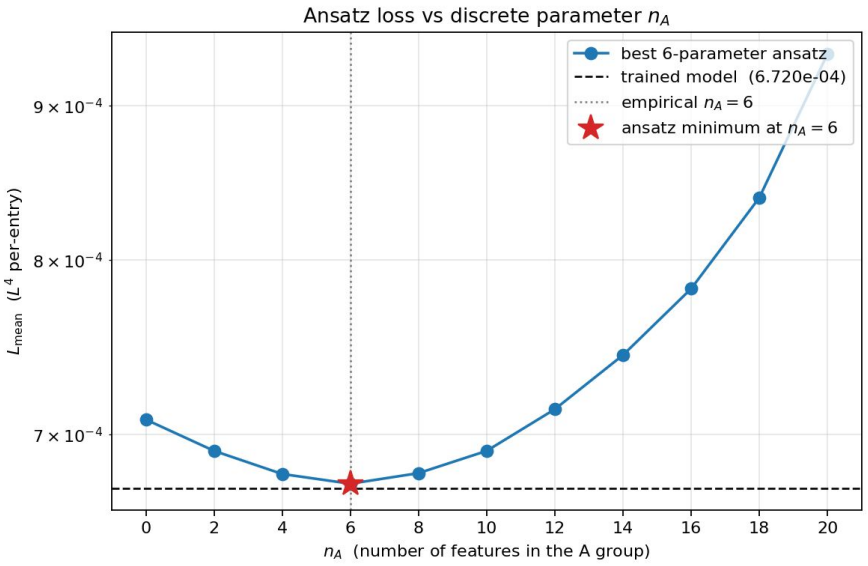


Claim: 6 numbers suffice to describe this model (down from 80)

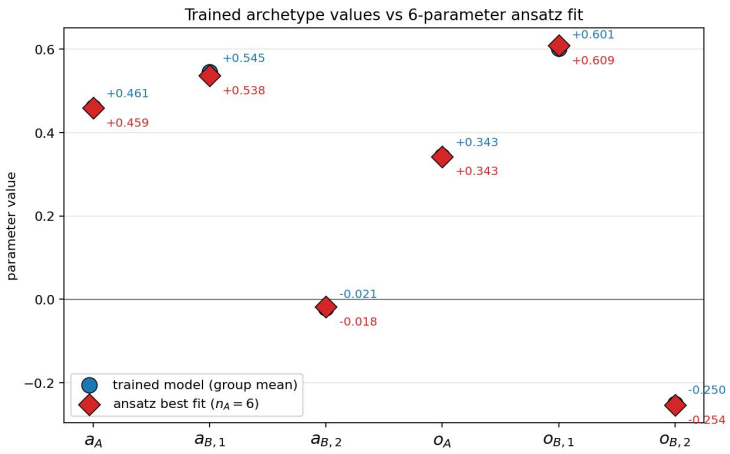
- A group both coords equal
- B <-> C symmetry
- W_out identical to this
- 1 DoF how many features per group

Three groups of features. Features within group treated identically

Training our ansatz model



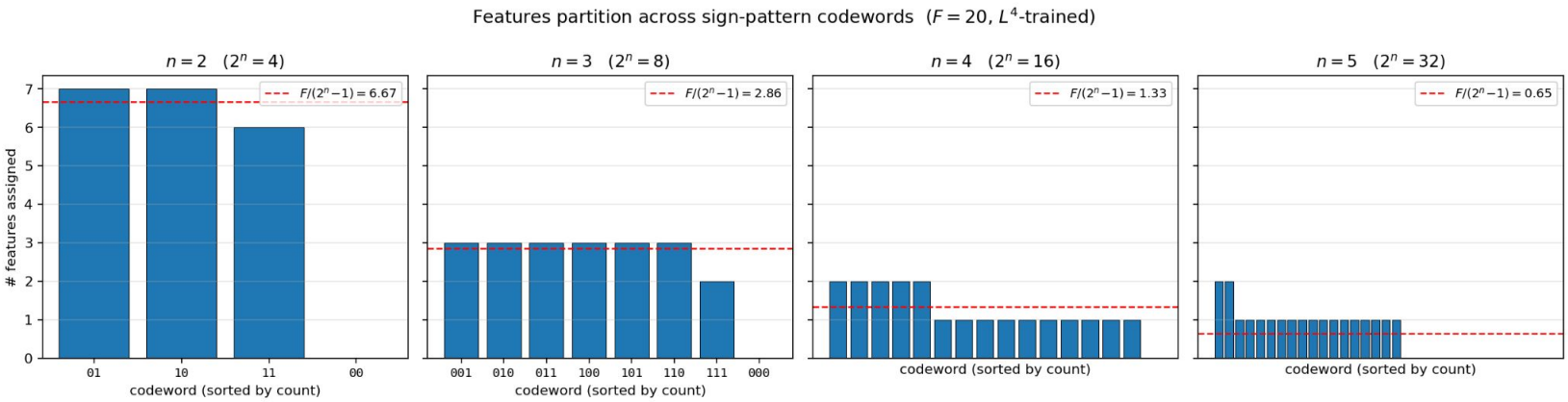
For given n_A, do gradient descent on L^4 loss to optimize other parameters. Best n_A is 6 (same as network), loss closely matches network's



Ansatz model parameters also fit network's

What does it mean, though?

Claim: Which neurons fire functions as binary code to identify active features
Prediction: For 3 neurons, 20 features, we get 6 codewords shared by 3 features, 1 by 2, etc.

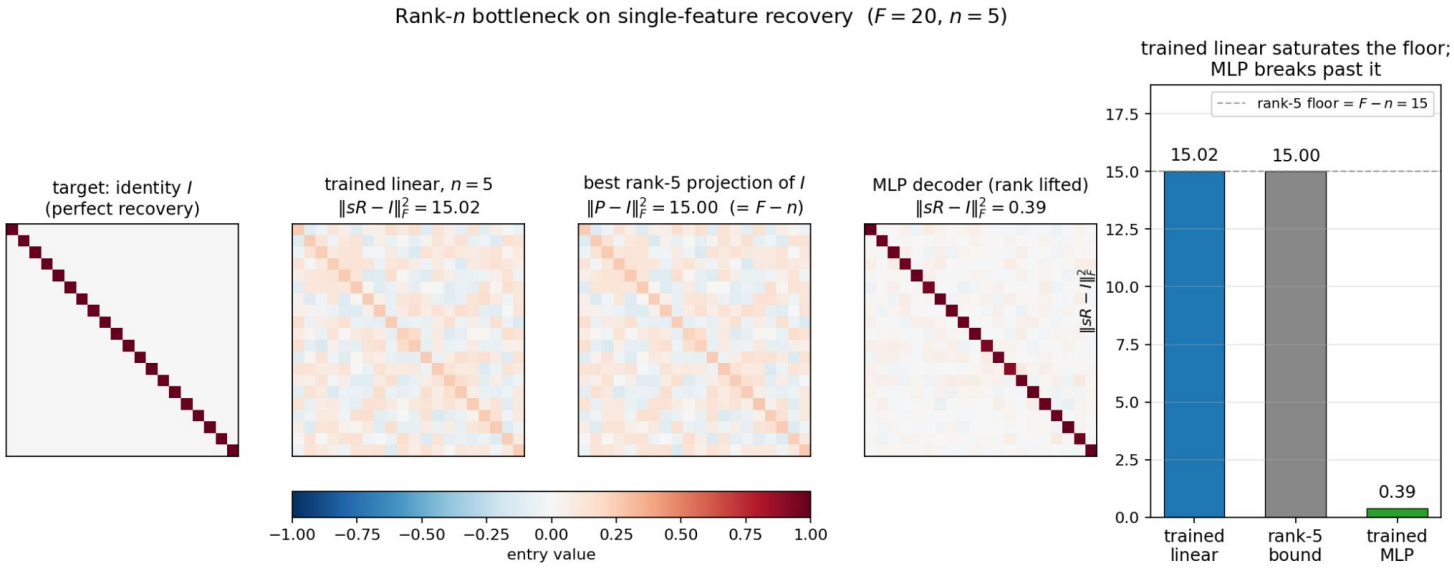


Two separate bottlenecks (single feature active)

Claim: We can identify two bottlenecks to this problem :

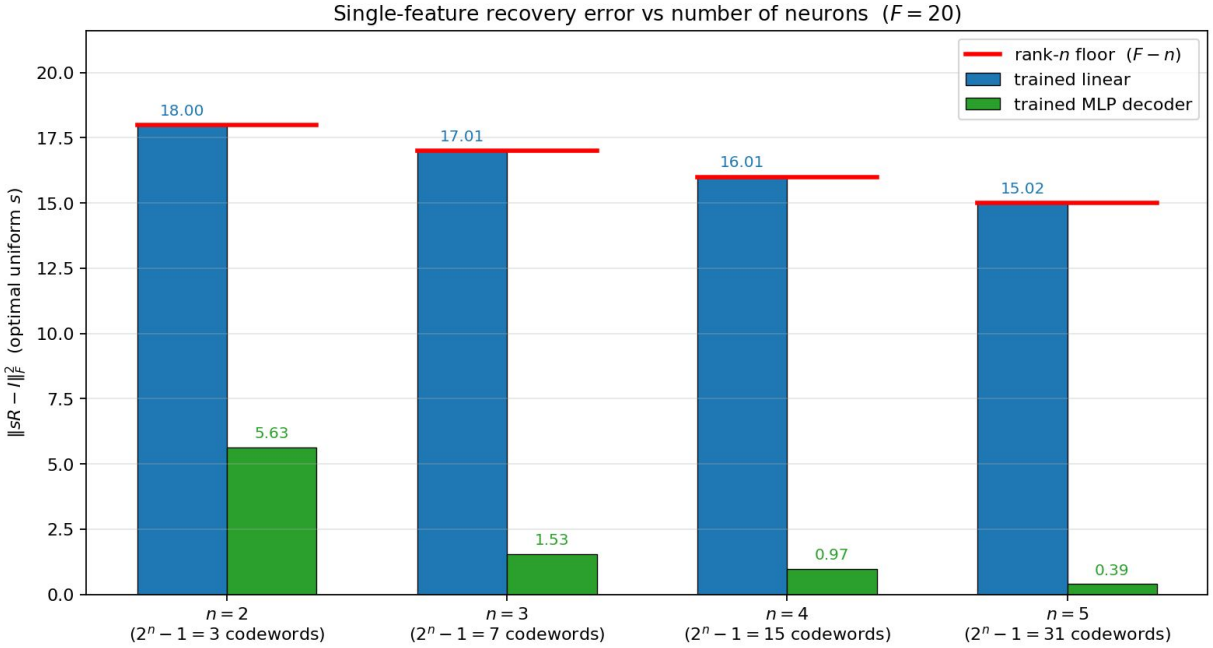
1. Identifying which feature is active
    a. Solved if n_features < 2^n_neurons
2. Recovering that feature's value
    a. Solved if n_features = 2_neurons OR
    b. Solved by non-linear decoder

Two separate bottlenecks - supporting evidence



- R is single-feature response
- s is the optimal uniform rescale
- P is rank-5 orthogonal projector in R^20

Two separate bottlenecks - supporting evidence



MLP decoder helps, but doesn't get close to zero if there's not enough codewords

What's next?

I have some ideas to try and strengthen my story:

1. Sweep exponent of loss. Can we identify exponent for which we get CiS-type solution?
2. Sweep sparsity. Can we identify regimes where different solutions work?
    a. Identifying which features fired via binary codewords works if there's few collisions only - should break down at higher sparsity
    b. Can we characterize code distance?
3. Do detailed ansatz for bigger model?

Happy to hear takes!

What's the goal with the post/paper?

- Outline for post along the lines 'Evidence against threshold-based error correction'
- Is this also the idea for the paper? Or more broadly, 'Here's what's wrong with MELBO-like methods'?